

Лабораторная работа №2

Цель работы

Исследовать структуры графических файлов и освоить чтение метаданных растровых изображений путем прямого побайтового парсинга файлов без использования сторонних библиотек.

Задачи

1. Изучить спецификации форматов растровых изображений: JPG, GIF, TIF, BMP, PNG, PCX.
2. Реализовать ручной парсинг заголовков, чанков и тегов файлов для извлечения основных свойств (имя файла, размер в пикселях, разрешение, глубина цвета, сжатие).
3. Оптимизировать приложение для работы со сверхбольшими папками (до 100 000 файлов) с использованием ленивого чтения и многопоточности.
4. Реализовать графический интерфейс пользователя (GUI) с табличным отображением результатов и прогресс-баром.
5. Реализовать валидацию файлов для детекции поврежденных (битых) изображений.

Использованные средства разработки

Язык программирования Python. Для организации масштабируемой архитектуры приложения использовано разделение на логические модули:

- `gui.py` — реализация графического интерфейса пользователя;
- `parser.py` — низкоуровневое чтение файлов и извлечение метаданных;
- `worker.py` — логика асинхронной и многопоточной обработки файлов;

Сборка проекта в единый исполняемый файл (exe) осуществлена через инструмент упаковки PyInstaller (конфигурация представлена файлом `main.spec`, артефакты в директориях `build/` и `dist/`).

Ход работы

1. Изучены спецификации и структуры целевых графических форматов: JPG, GIF, TIF, BMP, PNG, PCX. Были определены маркеры (Magic Numbers), сигнатуры и смещения тегов, необходимые для корректного извлечения метаданных из файловых заголовков.
2. В модуле `parser.py` реализован ручной побайтовый парсинг файлов без использования высокоуровневых сторонних библиотек (таких как PIL или OpenCV). Файлы открываются как поток байтов. Например, для формата PNG реализован поиск сигнатуры (89 50 4E 47) и чанка IHDR для считывания ширины, высоты и глубины цвета; для BMP — побайтовое чтение структур `BITMAPFILEHEADER` и `BITMAPINFOHEADER` для определения геометрии и типа сжатия.

3. Для обеспечения высокой скорости работы со сверхбольшими папками (содержащими файлы общим объемом в несколько гигабайт) в модуле `worker.py` реализована многопоточность. Применено ленивое (потокное) чтение: программа считывает только первые байты заголовка или прыгает по известным смещениям, вычисляет необходимые параметры и немедленно закрывает дескриптор, избегая полной загрузки графических данных в оперативную память.
4. Разработан отзывчивый графический интерфейс (GUI) в модуле `gui.py`. Интерфейс включает в себя диалог выбора директории, интерактивную таблицу для удобного представления считанной информации (имя, разрешение, глубина цвета, сжатие) и прогресс-бар. Для предотвращения блокировки (фризов) интерфейса во время дисковых операций вся нагрузка вынесена в фоновые потоки (Worker threads).
5. Реализована обработка граничных случаев и валидация файлов. Добавлена детекция поврежденных (битых) файлов или файлов с подмененным расширением. Если программа обнаруживает отсутствие маркера EOI (FF D9) в конце JPG-файла или отсутствие чанка IEND у PNG, выводится корректный статус «Файл поврежден», предотвращая аварийное завершение работы (NullPointerException, IndexOutOfRangeException) и гарантируя стабильность приложения.

Вывод

Практическим результатом выполнения лабораторной работы стало создание надежного и оптимизированного многопоточного настольного приложения на языке Python для чтения метаданных из графических файлов (JPG, GIF, TIF, BMP, PNG, PCX).

Строгое соблюдение архитектурного ограничения — отказ от сторонних библиотек в пользу ручного побайтового парсинга спецификаций (реализованного в модуле `parser.py`) — позволило глубоко понять внутреннее устройство различных графических форматов. Благодаря внедрению потокового чтения, асинхронных операций (`worker.py`) и отзывчивого интерфейса (`gui.py`), приложение успешно справляется со стресс-тестами и демонстрирует высокое быстродействие при обработке объемных директорий (до 100 000 файлов). Наличие валидации защищает программу от сбоев при столкновении с поврежденными файлами.